

solution for the contest

#程序设计竞赛

#算法组授课

首先，对愿意参加比赛的同学表示非常大的感谢。非常感谢大家能够百忙之中抽出时间，来做一做我们出的题目，并且坚持了下去。也希望大家能够继续对编程持有一份热爱，了解到算法竞赛。

其次，对于赛场中发生的一些状况，我们也表示抱歉，希望这些状况不会对您的成绩造成过大的影响。我们也尽力将问题做到最小化。

最后，恭喜获奖并获得优异成绩的各位。如果没有发挥好，不要灰心，迎接你的一定是一个更为广阔的明天。

为什么要演奏

传说中的，大家喜闻乐见的 `hello world` 题。只需会输出即可。如果你想追求速度，写如下代码即可。

```
print('32123432')
```

团子大家族

经典的 `a+b problem` 的弱化版 `a+114 problem`。

题目中还顺带了点皮亚诺公理（？）总之只要输出如下代码即可：

```
a = int(input())
print(str(a + 114))
```

恋爱配对

std

```
#include<stdio.h>
int a, b;

int main(){
    scanf("%d%d", &a, &b); // 输入编号
    if(a==13&&b==6||b==13&&a==6) printf("YES"); // 判断两人的 mbti 是否为官配
```

```

else if(a==1&&b==10||b==1&&a==10) printf("YES");
else if(a==7&&b==15||b==7&&a==15) printf("YES");
else if(a==9&&b==2||b==9&&a==2) printf("YES");
else if(a==12&&b==3||b==12&&a==3) printf("YES");
else if(a==14&&b==5||b==14&&a==5) printf("YES");
else if(a==11&&b==4||b==11&&a==4) printf("YES");
else if(a==8&&b==16||b==8&&a==16) printf("YES");
else printf("NO");
return 0;
}

```

易错点

- 因为分类较多，编号并非顺序，容易看错编号，写错判断条件
- 忘记考虑 mbti 编号 a 与 b 具有对称性，可以交换

骗分技巧

- 因为输出只有 YES 和 NO，所以直接输出 YES 或 NO 可以直接得 50 分

败犬太多了

考察 for 循环与 if 语句的运用。

我们使用 for 循环进行一次又一次的访问，在里面嵌套 if 语句即可。

完整代码如下：

```

#include<iostream>
using namespace std;

long long n;
long long a0,b0;

int main(){
    cin>>n>>a0>>b0;
    for(long long i = 1; i<=n;i++){
        long long a,b;
        cin>>a>>b;
        if(a > a0 && b < b0)cout<<"N"<<endl;
        else cout<<"Y"<<endl;
    }
    return 0;
}

```

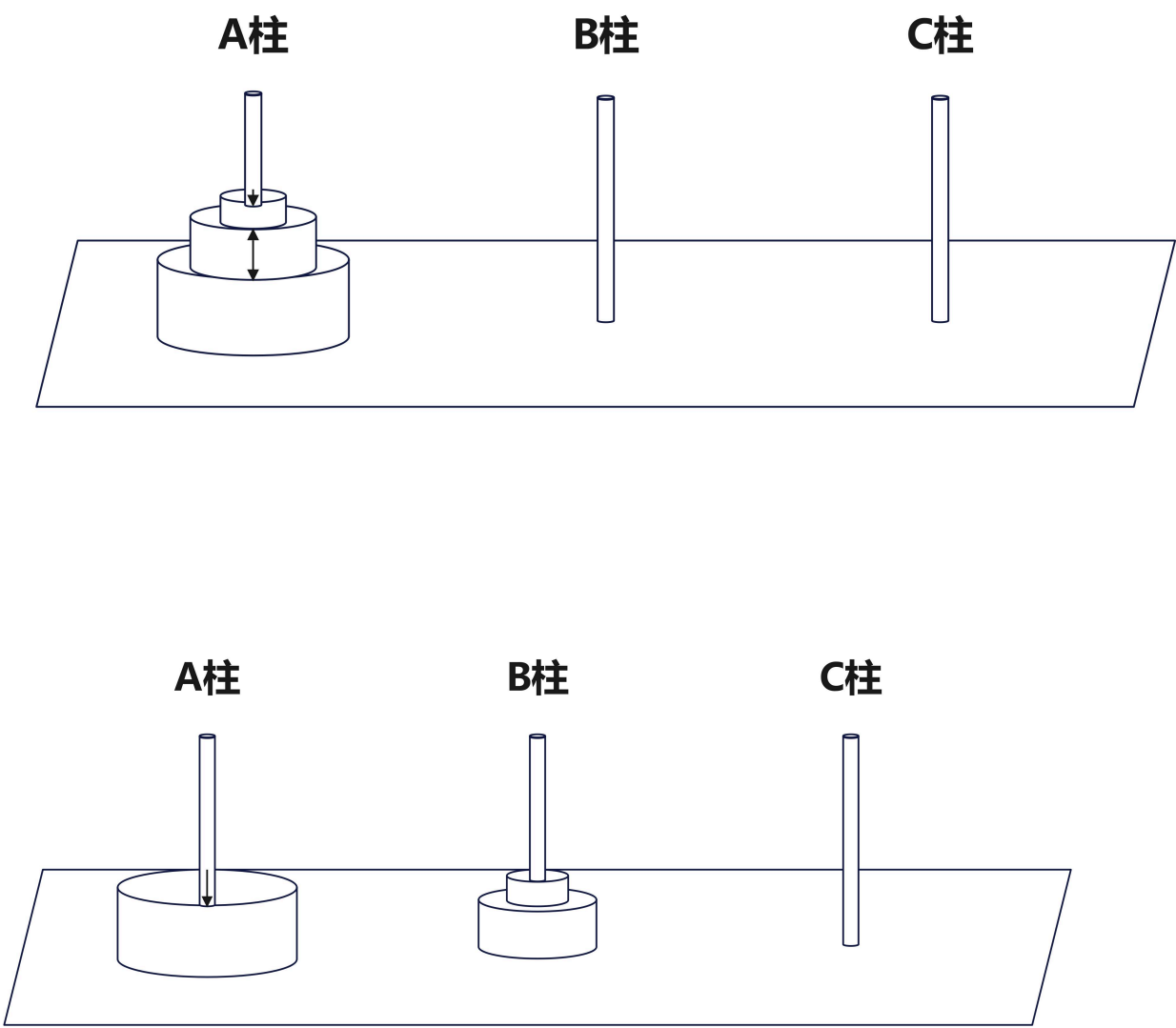
可怕的汉诺塔

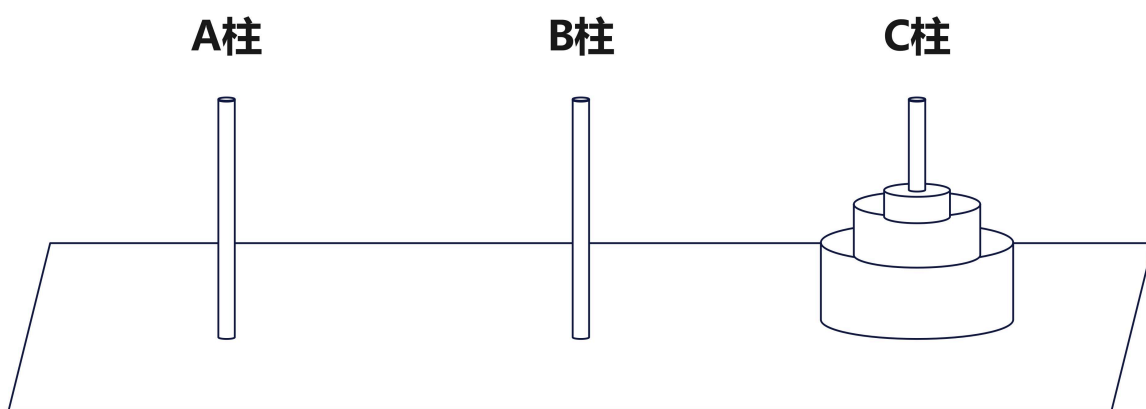
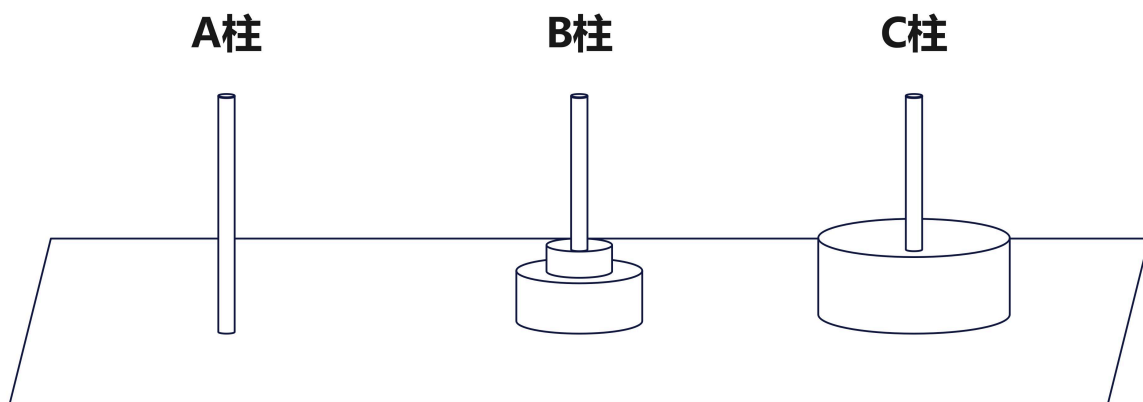
知识点：递推

设 $f(n)$ 表示将 n 个金片从A柱移动到C柱所需要的次数。

通过模拟我们可以发现，假设初始 n 个金片都在 A 柱上，我们需要先将 $n - 1$ 片移到 B 柱上，再将第 n 个金片移动到 C 柱上,最后将 B 柱上的 $n - 1$ 个金片移动到 C 柱上，完成操作。我们可以进一步发现，移动 $n - 1$ 个金片的过程又是相同的，实际整个过程是一个递推过程。

如图：





容易得出： $f(n) = 2 * f(n - 1) + 1, f(1) = 1$ 。

进一步对递推式进行化简，可以得出： $f(n) = 2^n - 1$

我太想进步了

几个需要考虑的坑点：

1. 最大值很好处理，但是如何处理第二大的值？
2. 有这么多的小数，应当用什么类型储存？

对于上面两个问题，其实第一个很简单，实现方式也很多样，可以先记录下最大值，然后再检验是不是这个最大值，如果不是就是次大值。

下面这个问题，我们毫无疑问要使用 `double` 类型储存。
代码如下：

```
#include<iostream>
#include<iomanip>
using namespace std;

long long n, m1, m2;
double goods[1000010];
double sum, max1, max2;

int main(){
    cin>>n;
    for(int i = 1; i <= n; i++){
        cin>>goods[i];
        sum += goods[i];
    }
    for(int i = 1; i <= n; i++){
        if(goods[i] > max1){
            max1 = goods[i];
            m1 = i;
        }
    }
    for(int i = 1; i <= n; i++){
        if(goods[i] > max2 && i != m1){
            max2 = goods[i];
            m2 = i;
        }
    }
    cout<<fixed<<setprecision(2)<<sum<<endl;
    cout<<m1<<" "<<m2<<endl;
    return 0;
}
```

减跑操

考点

`if` 条件判断、`for/while` 循环

题意分析

根据题意可知我们要实现以下功能：

- 输入 n
- 循环遍历 $1 \sim n$ 的所有数字
- 判断数字是否含 7 或是否是 7 的倍数
- 输出数字或 *

std

```
#include<stdio.h>
int n;
int main(){
    scanf("%d", &n); // 输入 n
    for(int i=1; i<=n; i++){ // 遍历 1~n 的所有数字
        if(i%7==0) printf("*"); // 判断数字是否是 7 的倍数, 如果是 7 的倍数, 输出 *
        else{
            int j=i, op=0; // op 做标记, 0 为数字不含 7, 1 为数字含 7
            while(j){ // 判断数字是否含 7
                if(j%10==7){ // 如果含 7, 输出 *
                    printf("*");
                    op=1;
                    break;
                }
                j=j/10;
            }
            if(op==0) printf("%d", i); // 如果不含, 输出数字
        }
        if(i!=n) printf(" ");
    }
    return 0;
}
```

易错点

- 未判断数字是否含 7 或是否是 7 的倍数的其中一个
- 输出时未包括 n

维吉尼亚密码

知识点：字符串基本操作

补全明文和密文长度，或者通过求余运算使得明文能与密文对应字母对上。
由图表关系，找出字母间的规律：

$L[i]$ 表示密文中第 i 个字符
 $S[i]$ 表示明文的第 i 个字符
 $P[i]$ 表示密钥的第 i 个字符
可以得出密文与明文和密钥的关系：
 $L[i] = 'A' + (S[i] - 'A' + P[j] - 'A') \% 26;$

布洛妮娅的绀海之约

题意就是求有多少个四元组出现次数大于2次。

解法一：考虑使用map+结构体进行操作，需要注意应该map内存结构体需要对<号进行重定向。

解法二：进行结构体排序，从而分成很多不同的块，统计大小大于2的块的个数。

落入深海的少女

题意简化一下就是对于一个二分图，二分图之间有边权，求得一种匹配方案，使得匹配边边权和最小。

解法一：对于二分图最大权匹配，使用KM算法。但是由于KM做的是最大权匹配，因此需要在一开始将边权取反。

解法二：网络流建图跑费用流，源点向量子点连边，虚数点向汇点连边，流量上限均为1，费用为0。量子点和虚数点间的边流量为1，费用为边权，跑最小费用最大流。

解法三(std做法)：观察发现 n 的值比较小，且保证了数据随机，使用暴力。我们先构建任意一个排列 r_i ，表示第 i 个量子点和第 r_i 个虚数点匹配。之后 n^2 枚举 (i, j) ，判断将 r_i, r_j 交换后，会不会是答案变小，如果会，就进行交换。重复执行，直至某次发现没有交换发生，表示已经找到最优答案。

理论上最坏情况是每次只交换一对点，因此最坏复杂度为 $O(n^4)$ ，但由于数据随机生成，因此暴力次数实际上并不会超过 n ，所以在随机数据下，复杂度甚至是远小于 $O(n^3)$ 的。

拟造，月光王座

假设我们需要构建 m 台王座。可以发现一个很显然的性质，我们使用扫描操作时，场上的王座数量一定是 m 的约数。

因此我们先使用 $O(\sqrt{n})$ 的做法跑出 m 的所有约数。

令 f_i 表示构建出 i 个王座需要花费的最小能量。初始情况 $f_i = k + i - 1$

考虑转移，枚举 i 的所有约数 j ， $f_i = \min(f_i, f_j + f_{i/j})$ ，举个例子，我们需要构建12，可以先构建6，然后对6构建2，也就得到了 $6 \times 2 = 12$ 。

所以最后的 f_n 就是答案

鸭鸭和哄姆

显然，我们最多只会询问一次，并且在一开始询问一定是最优的。

由于我们在一开始询问了存在的哄姆的总个数，并且之后我们打开某个箱子就能知道该箱子里有没有哄姆，也就变相知道了剩下没打开的箱子里有多少个哄姆，因此没必要白花钱询问。

考虑将 w_i 排个序， w 越大的我们肯定要越后面打开。

接下来考虑是否询问，如果我们选择不询问，那么我们想要知道所有箱子的情况，只能打开所有箱子。答案就是所有 w 的和。

如果询问了，我们考虑什么情况下能知道所有箱子是否有哄姆。

当且仅当通过计算得出的剩下箱子中哄姆玩偶个数为0或者个数等于箱子个数时，我们可以不必继续开箱。

因此，无论哄姆如何分布，最后一个箱子一定不会被打开。那么倒数第 i 个箱子需要被打开的概率是多少呢？我们用0表示无哄姆，1表示有哄姆，当且仅当全0或者全1时可以不用打开，那么打开的概率就是 $1 - \frac{1}{2^{i-1}}$ 。

将排序后每个 w 乘上对应被打开的概率加上询问一次的代价就是答案，别忘了和 w 的总和进行比较。

健身

知识点：贪心，动态规划

解法一：贪心，暴力模拟(10~20分)

算出每个计划性价比（健身收益/天数），进行排序。根据有安排的日子，将空余时间段分成若干个区间，对每个区间依照性价比求解其最优答案，最后求出最终答案。

解法二:动态规划

由于健身计划时间为 2^i ，且任何正整数都可以由

设 $f(i)$ 表示前 i 天的最大健身收益，

$f(i) = \max\{f(i - 2^{k_j}) + s_j, i - 2^{k_j} \leq 0 \text{ 且 } G(i - 2^{k_j}, i) = 1\}$, $G(l, r) = 1$ 表示在 $[l, r]$ 内没有其他安排。

给出一种 $O(1)$ 时间内判断 $G(l, r) == 1$ 的方法：

前缀和：

设 $d[i]$ 表示前 i 天共有多少天有安排,若 $d[r] == d[l]$,则可认为在这段区间内没有其他任务安排。

例如:

day	1	2	3	4	5	6
d[i]	0	1	1	2	2	2

解法三：对解法二的优化

我们发现其实我们要求的区间是一段一段的，我们可以预处理出所有所需天数的最优答案，该过程其实是个 **完全背包问题**。设 $f(i)$ 表示 i 天时间用来健身的最大收益,这样我们就不需要判断有安排的日子。

设区间的段数为 T ，每段区间的长度为 len_i

最终 $ans = \sum_{i=1}^T f(len_i)$

生物的世界

简化题意：给一个有根树，判断它有多少条最长链。

解释：其实题目对“生物”，“骨骼”的描述都是在建图，这就遇到了图论的相关知识。

因为对于 n 个点，只有 $n - 1$ 条边，而且图是联通的，所以它就是一棵树，也就是说，他是无环的。

选取一个点作为树的根，然后以此根为起始点，不断地深度/广度优先搜索，直到搜到叶子节点结束，记录这一串链的长度。，更新最大链长度，记录长链个数。

下面给出深度优先搜索核心代码，非常简单(我们采用邻接表来储存图)：

```
void dfs(int v){
    for(auto i : g[v]){
        if(vis[i])continue;
        dis[i] = dis[v] + 1;
        vis[i] = 1;
        dfs(i);
    }
}
```

冰箱

神奇大模拟题目。题目的难点在于理解题目，写出代码。

圣芙蕾雅

首先求出每两位女武神是否可以分配同一套装甲，即判断是否存在一个事件（点）上，这两女武神都已经分配了装甲，且后续都有可能参加战斗。

枚举每个女武神 x ，从每个对应战斗事件开始向前遍历，遇到对应分配事件停止。途中遇到的每个非对应分配事件，可得到一条冲突关系：女武神 x 与此分配事件对应的女武神 y 不能分配同一套装甲，因为此时女武神 x 已分配装甲且后续有可能参加战斗，而女武神 y 此时要被分配装甲。

令 D_x 表示女武神 x 对应的分配事件。

- 性质1：如果在事件 u 结束后，女武神 x 依然可能参加战斗（即从点 u 可以到达 x 对应的战斗事件节点，下同），则 D_x 支配 u ，否则不满足题意。
- 性质2：如果在事件 u 结束后，女武神 x, y 都依然可能参加战斗，则 D_x 支配 D_y 或 D_y 支配 D_x 。

证明：根据性质1，有 D_x 支配 u 且 D_y 支配 u ，而支配关系为树形结构，因此得到 D_x 支配 D_y 或 D_y 支配 D_x 。

- 性质3：如果女武神 x, y 不能分配同一套装甲，且 D_x 支配 D_y ，则在 D_y 处，女武神 x 依然有可能参加战斗。

证明：如果 D_y 处女武神 x 已经一定不会参加战斗了，那么可以和 y 分配同一套装甲。

- 性质4：如果女武神 x, y 不能分配同一套装甲，女武神 y, z 不能分配同一套装甲，但是女武神 x, z 可以分配同一套装甲，且 D_x 支配 D_y ，则 D_y 支配 D_z 。

证明：由于 y, z 不能分配同一套装甲，因此 D_y 支配 D_z 或 D_z 支配 D_y 。假设 D_z 支配 D_y ，则 z 在 D_y 处依然可能参加战斗，同理，由于 x 在 D_y 处依然可能参加战斗，则 x 与 z 不能分配同一套装甲，与前提矛盾，因此 D_y 支配 D_z 。

结论：女武神间根据不能分配同一套装甲的关系建立无向图，此图一定是弦图。

证明：假设不是弦图，则一定存在长度大于等于4的无弦环，不妨设

其为 $x_1, x_2, \dots, x_{n-1}, x_n$ ，其中 $n \geq 4$ 。由于 x_1, x_2 不能分配同一套装甲，不失一般性，假设 D_{x_1} 支配 D_{x_2} ，反复应用性质4，可以得到假设 D_{x_1} 支配 D_{x_2} ， D_{x_2} 支配 D_{x_3} ， $\dots$ ， $D_{x_{n-1}}$ 支配 D_{x_n} 。

由于 D_{x_1}, D_{x_n} 不能分配同一套装甲，因此存在一个事件 u ，在 u 结束时这两女武神都依然有可能参加战斗。根据性质1得 D_{x_n} 支配 u ，则从 $D_i (1 \leq i \leq n)$ 都可以到达 u ，因此 x_1 在这些事件处都依然有可能参加战斗，即 x_1 与 $x_i (2 \leq i \leq n)$ 都不能分配同一套装甲，与前提矛盾。

因此后续问题即为在弦图上求最小染色数，用完美消除序列求解即可。

事实上也可以不使用完美消除序列的构造算法，对原图求支配树，可以根据支配树的后序遍历来得到完美消除序列。时间复杂度 $O(k(n + m))$